



FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING)

WITH HONOURS

Microservice-Based Disaster Relief Coordination Platform Proposal

(Group 1)

Submitted To:

Dr. Mohamad Nor Hasan

NAME	MATRIC NO.
SATRIALDY DARREL FARIZKI	S76351
FLORA TANG SUE QI	S74646
MOHAMED ASHRAFF BIN MOHAMED MUHRIZAN	S62368

1. Selected Architecture Style: Microservices

For a Disaster Relief Coordination Platform, the system must remain stable, and highly available under extreme conditions. To meet these demands, our team has selected the **Microservices Architecture** style. Unlike a monolithic approach, this architecture breaks down the project into independent services, each running its own process.

1.1 Why Microservices?

- **High Availability and Fault Isolation:** In a disaster scenario, it is paramount that the failure of one component must not bring down the entire platform. Microservices ensure that if a non-critical service fails, the core functionality remains operational.
- **Flexible Scalability:** Disaster events cause sudden, massive spikes in user traffic. Microservices allow us to scale specific high-demand services independently. For example, during a flood, the Incident Service can be scaled across multiple servers and containers to handle thousands of incoming reports without needing to scale the entire administrative backend.
- **Independent Deployment and Evolution:** Relief efforts require rapid updates as new needs arise. A microservices style allows our team to develop, test, and deploy new features without redeploying the entire system, ensuring continuous service delivery.

1.2 Proposed Tech-Stack

1.2.1 Frontend & API Management

- **Frontend Framework: React** React will drive the user interface, its efficient state management is needed for rendering real-time updates for field responders and administrators.
- **Web Server: Nginx** Nginx will serve the compiled React application.
- **API Gateway: Kong** Acting as the central entry point for all client requests, Kong will manage routing, load balancing, rate limiting, and initial security validations.

1.2.2 Core Backend Services

- **Microservices Framework: Quarkus** The backend services will be developed using Quarkus, a Kubernetes-native Java framework. Quarkus is selected for its fast boot times and low memory consumption, making it ideal for running multiple microservices simultaneously in a local environment with lower than average resources.

1.2.3 Data Persistence & Caching

- **Primary Database: PostgreSQL** PostgreSQL will serve as the primary database for services requiring strict data integrity and complex relational mapping (such as the Incident and Resource Services).
 - *Geospatial Extension:* The **PostGIS** extension will be integrated into PostgreSQL to power the **GIS / Map Service**.
- **In-Memory Data Store: Redis** Redis will handle high-speed data retrieval and caching. It is essential for managing ephemeral, fast-paced data for the **Notification Service**, such as active WebSocket sessions and real-time geo-fenced alert states.

1.2.4 Identity & Access Management (IAM)

- **Authentication & Authorization: Keycloak** Keycloak will provide the Identity Service to handle OAuth2/JWT token issuance, user federation, and Role-Based Access Control (RBAC).

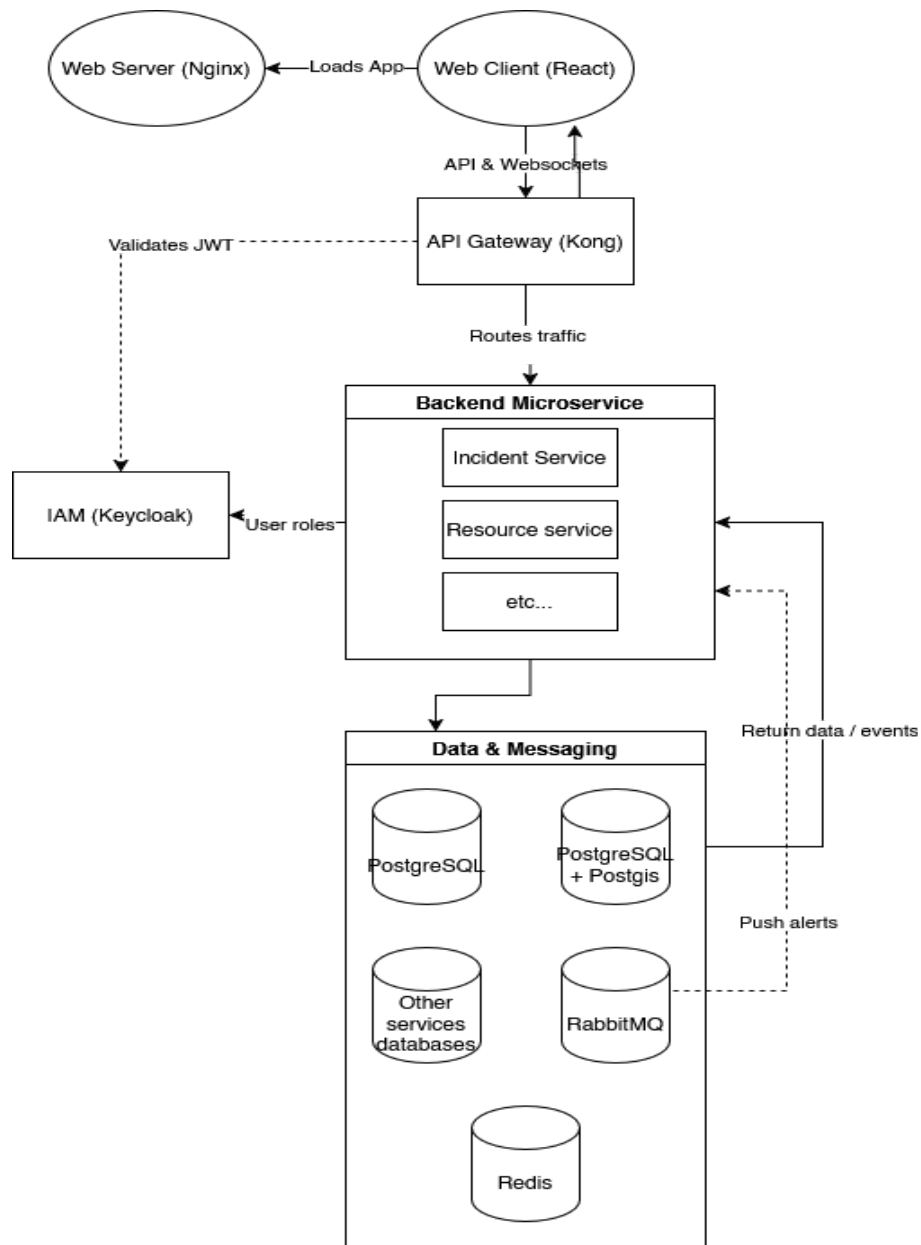
1.2.5 Event-Driven Communication

- **Message Broker: RabbitMQ** To prevent tight coupling between microservices, RabbitMQ will facilitate asynchronous, event-driven communication. For example, when the Incident Service logs a new disaster, it will publish an event to RabbitMQ, which the Notification Service will consume to broadcast real-time alerts without causing system bottlenecks.

1.2.6 Infrastructure & Local Environment

- **Containerization & Orchestration: Docker & Docker Compose** The entire application ecosystem, including all Quarkus services, the React frontend, Kong, databases, Keycloak, and RabbitMQ will be containerized using Docker. Docker Compose will manage local orchestration.

1.3 Proposed System Diagram



2. Problem Description

Disasters happen unexpectedly, and during these times, it's crucial that help reaches those in need as quickly as possible. However, many existing disaster relief systems struggle with slow responses, poor communication, and difficulties in tracking resources. Our project aims to solve these problems by creating a Disaster Relief Coordination Platform based on microservices. This system will be broken down into smaller, independent parts that can work together while still functioning on their own. This means if one part of the system fails, the rest will continue working, ensuring no interruptions when disaster strikes. Whether it's victims needing help, responders providing aid, or administrators organizing the relief, our platform will make everything run smoothly, quickly, and efficiently.

3. System Objectives

1. **Stay Available in Tough Times:** The system must be reliable, so if one part breaks down, the rest will still work. This ensures critical functions, like reporting incidents or tracking resources, are always available.
2. **Instant Communication:** The platform will allow real-time messages and alerts between victims, responders, and administrators, helping everyone stay connected and up-to-date during a disaster.
3. **Handle High Traffic:** Disasters can bring lots of people using the platform at once. The system needs to handle this extra demand without slowing down. Microservices allow us to increase certain parts of the system, like incident reporting, as needed.
4. **Quick Updates:** When new needs arise, we want to quickly update the platform, such as adding a new feature for medical help. The system allows us to make these changes without shutting everything down.
5. **Track Resources and Locations:** The platform will include maps and tools to see disaster areas and manage relief supplies. This way, responders can easily see where help is needed and where supplies are located, making the relief effort faster and more organized.

6. **Keep It Secure:** The platform will have strong security features to make sure only the right people can access sensitive information and perform important tasks. Victims, responders, and administrators will each have the appropriate access based on their role.

4. Initial Functional Requirements

To ensure the system meets the needs of various stakeholders, the requirements are mapped to three primary user roles: **Victims** (citizens in need of aid), **Responders** (field workers/emergency personnel), and **Administrators** (coordinators and agency leads).

4.1 Architectural Service Overview

Service Name	Primary Responsibility	Key Technical Features
Identity Service	Manages authentication, authorization, and role management.	OAuth2/JWT implementation, RBAC (Role-Based Access Control), and responder verification.
Incident Service	Handles the lifecycle of disaster reports and field data.	Geolocation tagging, image attachment support, and real-time status tracking.
Resource Service	Manages the inventory and logistics of relief supplies.	Stock updates, supply-chain tracking, and automated request handling.
GIS/Map Service	Provides geospatial visualization and coordination tools.	Heatmap generation, interactive mapping (Leaflet/Mapbox), and proximity-based routing.
Notification Service	Facilitates real-time communication and emergency alerts.	Push notifications, WebSocket for live chat, and geo-fenced broadcasting.

4.2 Summary of Functional Requirements by User Role

Requirement Category	Victim	Responder	Administrator
Report Incident	Yes	Yes	Yes
View Resource Map	Limited	Full	Full

Manage Inventory	No	No	Yes
Update Incident Status	No	Yes	Yes
Broadcast Alerts	No	No	Yes
Edit User Roles	No	No	Yes

5. Team Member Roles

Name	Roles	Responsibilities
SATRIALDY DARREL FARIZKI	Project Lead and System Architect	- Leads the project and coordinates team activities and timeline. - Designs the overall system architecture Modules : Identity Service, Incident Service
FLORA TANG SUE QI	Backend Developer	- Develops core backend services, including incident management, resource allocation, and notification systems. Modules: Resource Service, GIS/MAP Service
MOHAMED ASHRAFF BIN MOHAMED MUHRIZAN	Frontend Developer	- Develops the user interface and ensures a smooth user experience. - Integrates the API provided by the backend services to the frontend ensuring smooth communication Modules: Notification Service, UI/UX Design, Frontend Integration